

Static Application Security Testing (SAST) Audit Report

Scanner Model: No LLM model used; deterministic heuristic SAST rules
Detected Languages: .NET/C#, JavaScript

Executive Summary

Total Vulnerabilities Identified: 33

Severity Level	Identified Count
Critical	2
High	31
Medium	0
Low	0

Detailed Security Findings

1. OS Command Injection via Runtime / Shell Execution - CRITICAL

CWE / Taxonomy:	CWE-78 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/jquery-3.3.1.slim.min.js (Lines 2-2)

Focused Vulnerable Code:

```
...n++]=t[r++]);e.length=n-1}}function oe(e,t,r,i){var
o,u,l,c,f,h,y,m=t&&t.ownerDocument,C=t?t.nodeType:9;if(r=r||[],"string"!=typeof
e||!e||1!=C&&9!=C&&11!=C)return r;if(!i&&((t?t.ownerDocument||t:w)!=p&&d(t),t=t||p,g)){if(11==C
&&(f=K.exec(e)))if(o=f[1]){if(9==C){if(!(l=t.getElementById(o)))return r;if(l.id==o)return
r.push(l,r)}else if(m&&(l=m.getElementById(o))&&b(t,l)&&l.id==o)return
r.push(l,r)}else{if(f[2])return q.apply(r,t.getElementsByTagName(e)),r;if((o=f[3])...
```

Copy/Paste Fix:

```
execFile(command, [argument], { shell: false }, callback);
```

Remediation Strategy: Avoid invoking system shells directly. Parameterize arguments using structured array APIs.

References:

<https://cwe.mitre.org/data/definitions/78.html>
https://cheatsheetseries.owasp.org/cheatsheets/Injection_Prevention_Cheat_Sheet.html

2. OS Command Injection via Runtime / Shell Execution - CRITICAL

CWE / Taxonomy:	CWE-78 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/jquery-3.3.1.min.js (Lines 2-2)

Focused Vulnerable Code:

```
...n++] = t[r++]; e.length = n - 1}}function oe(e, t, r, i){var
o, s, l, c, f, h, v, m = t&&t.ownerDocument, T = t? t.nodeType: 9; if (r = r || [], "string" != typeof
e || !e || 1 !== T && 9 !== T && 11 !== T) return r; if (!i && ((t? t.ownerDocument | | t: w) !== d && p(t), t = t | | d, g)){if (11 !== T
&& (f = j.exec(e))) if (o = f[1]){if (9 === T){if (!(l = t.getElementById(o))) return r; if (l.id === o) return
r.push(l, r)} else if (m && (l = m.getElementById(o)) && x(t, l) && l.id === o) return
r.push(l, r)} else if (f[2]) return L.apply(r, t.getElementsByTagName(e)), r; if ((o = f[3])...
```

Copy/Paste Fix:

```
execFile(command, [argument], { shell: false }, callback);
```

Remediation Strategy: Avoid invoking system shells directly. Parameterize arguments using structured array APIs.

References:

<https://cwe.mitre.org/data/definitions/78.html>

https://cheatsheetseries.owasp.org/cheatsheets/Injection_Prevention_Cheat_Sheet.html

3. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/bootstrap.js (Lines 1314-1314)

Focused Vulnerable Code:

```
throw new Error(`selector` option must be specified when initializing ' + this.type + ' on the
window.document object!')
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

4. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/bootstrap.js (Lines 1464-1464)

Focused Vulnerable Code:

```
this.$element.trigger('inserted.bs.' + this.type)
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

5. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/bootstrap.js (Lines 1906-1906)

Focused Vulnerable Code:

```
this.selector = (this.options.target || '') + ' .nav li > a'
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

6. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/bootstrap.js (Lines 1995-1995)

Focused Vulnerable Code:

```
var selector = this.selector +
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

7. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/bootstrap.js (Lines 1997-1997)

Focused Vulnerable Code:

```
this.selector + '[href="' + target + '"]'
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

8. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/jquery-3.3.1.js (Lines 794-794)

Focused Vulnerable Code:

```
!compilerCache[ selector + " " ] &&
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

9. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/jquery-3.3.1.js (Lines 1249-1249)

Focused Vulnerable Code:

```
"<select id='" + expando + "' msallowcapture='>" +
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

10. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/jquery-3.3.1.js (Lines 1267-1267)

Focused Vulnerable Code:

```
if ( !el.querySelectorAll( "[id~=" + expando + "]" ).length ) {
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

11. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/jquery-3.3.1.js (Lines 1281-1281)

Focused Vulnerable Code:

```
if ( !el.querySelectorAll( "a#" + expando + "+" ).length ) {
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

12. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/jquery-3.3.1.js (Lines 2134-2134)

Focused Vulnerable Code:

```
cached = tokenCache[ selector + " " ];
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

13. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/jquery-3.3.1.js (Lines 2203-2203)

Focused Vulnerable Code:

```
selector += tokens[i].value;
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

14. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/jquery-3.3.1.js (Lines 2582-2582)

Focused Vulnerable Code:

```
cached = compilerCache[ selector + " " ];
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

15. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/jquery-3.3.1.js (Lines 5234-5234)

Focused Vulnerable Code:

```
sel = handleObj.selector + " ";
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

16. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/jquery-3.3.1.js (Lines 7952-7952)

Focused Vulnerable Code:

```
className = " " + selector + " ";
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

17. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/jquery.validate.js (Lines 429-429)

Focused Vulnerable Code:

```
":text, [type='password'], [type='file'], select, textarea, [type='number'], [type='search'], " +
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

18. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/jquery.validate.js (Lines 1015-1015)

Focused Vulnerable Code:

```
selector = "label[for='" + name + "'], label[for='" + name + "'] *";
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

19. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/jquery.validate.js (Lines 1019-1019)

Focused Vulnerable Code:

```
selector = selector + ", #" + this.escapeCssMeta( describer )
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

20. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/jquery-3.3.1.slim.min.js (Lines 2-2)

Focused Vulnerable Code:

```
...[c.call(e)]|"object":typeof e}var x="3.3.1 -ajax,-ajax/jsonp,-ajax/load,-ajax/parseXML,-ajax/sc  
ript,-ajax/var/location,-ajax/var/nonce,-ajax/var/rquery,-ajax/xhr,-manipulation/_evalUrl,-event/aja  
x,-effects,-effects/Tween,-effects/animatedSelector",w=function(e,t){return new  
w.fn.init(e,t)},C=/^[\s\uFEFF\xA0]+|[\s\uFEFF\xA0]+$|g;w.fn=w.prototype={jquery:x,constructor:w,leng  
th:0,toArray:function(){return o.call(this)},get:function(e){return  
null==e?o.call(this):e<0?this[e+thi...
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

21. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/jquery-3.3.1.min.js (Lines 2-2)

Focused Vulnerable Code:

```
...ew Date,w=e.document,T=0,C=0,E=ae(),k=ae(),S=ae(),D=function(e,t){return
e===t&&(f=!0),0},N={}.hasOwnProperty,A=[],j=A.pop,q=A.push,L=A.push,H=A.slice,O=function(e,t){for(va
r n=0,r=e.length;n<r;n++)if(e[n]===t)return n;return-1},P="checked|selected|async|autofocus|autoplay
|controls|defer|disabled|hidden|ismap|loop|multiple|open|readonly|required|scoped",M="[\\x20\\t\\r\\
n\\f]",R="(?:\\\\\\\\.|[\\w-]|[^\\0-\\xa0])+",I="\\["+M+"*("+R+")"(?:"+M+"*([*^$|!~]?=)"+M+"*"?:(?:(?:\\\\\\\\.
|[\\w-]|[^\\0-\\xa0])+"")?)[^\\]]*"
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

22. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/jquery.validate.min.js (Lines 4-4)

Focused Vulnerable Code:

```
...e.amd?define(["jquery"],a):"object"===typeof module&&module.exports?module.exports=a(require("jqe
ry")):a(jQuery)}(function(a){a.extend(a.fn,{validate:function(b){if(!this.length)return
void(b&&b.debug&&window.console&&console.warn("Nothing selected, can't validate, returning
nothing."));var c=a.data(this[0],"validator");return
c?c:(this.attr("novalidate","novalidate"),c=new a.validator(b,this[0]),a.data(this[0],"validator",c),
c.settings.onSubmit&&(this.on("click.validate","s...
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

23. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/bootstrap.min.js (Lines 6-6)

Focused Vulnerable Code:

```

...ction(d){var e=a(this);if(!e.is(".disabled, :disabled")){var
f=b(e),g=f.hasClass("open");if(c(),!g){ontouchstart"in
document.documentElement&&!f.closest(".navbar-nav").length&&a(document.createElement("div")).addClas
s("dropdown-backdrop").insertAfter(a(this)).on("click",c);var
h={relatedTarget:this};if(f.trigger(d=a.Event("show.bs.dropdown",h)),d.isDefaultPrevented())return;e.
trigger("focus").attr("aria-expanded","true"),f.toggleClass("open").trigger(a.Event("shown.bs.dropdo.
..

```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

24. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/bootstrap.min.js (Lines 7-7)

Focused Vulnerable Code:

```

this.activeTarget=b,this.clear();var c=this.selector+'[data-target="'+b+'"]','+this.selector+'[href="'
+b+'"]',d=a(c).parents("li").addClass("active");d.parent(".dropdown-menu").length&&(d=d.closest("li.
dropdown")).addClass("active"),d.trigger("activate.bs.scrollspy")),b.prototype.clea...

```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

25. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/jquery-3.3.1.slim.js (Lines 794-794)

Focused Vulnerable Code:

```
!compilerCache[ selector + " " ] &&
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

26. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
------------------------	-----------------------------

File Location:	VulnerableApplication/Scripts/jquery-3.3.1.slim.js (Lines 1249-1249)
-----------------------	--

Focused Vulnerable Code:

```
"<select id='" + expando + "-\r\\' msallowcapture=''" +
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

27. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/jquery-3.3.1.slim.js (Lines 1267-1267)

Focused Vulnerable Code:

```
if ( !el.querySelectorAll( "[id~=" + expando + "]" ).length ) {
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

28. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/jquery-3.3.1.slim.js (Lines 1281-1281)

Focused Vulnerable Code:

```
if ( !el.querySelectorAll( "a#" + expando + "+" ).length ) {
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

29. Potential SQL Injection via String Concatenation - HIGH

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/jquery-3.3.1.slim.js (Lines 2134-2134)

Focused Vulnerable Code:

```
cached = tokenCache[ selector + " " ];
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

30. Potential SQL Injection via String Concatenation - **HIGH**

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/jquery-3.3.1.slim.js (Lines 2203-2203)

Focused Vulnerable Code:

```
selector += tokens[i].value;
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

31. Potential SQL Injection via String Concatenation - **HIGH**

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/jquery-3.3.1.slim.js (Lines 2582-2582)

Focused Vulnerable Code:

```
cached = compilerCache[ selector + " " ];
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

32. Potential SQL Injection via String Concatenation - **HIGH**

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/jquery-3.3.1.slim.js (Lines 5234-5234)

Focused Vulnerable Code:

```
sel = handleObj.selector + " ";
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html

33. Potential SQL Injection via String Concatenation - **HIGH**

CWE / Taxonomy:	CWE-89 (A03:2021-Injection)
File Location:	VulnerableApplication/Scripts/jquery-3.3.1.slim.js (Lines 7159-7159)

Focused Vulnerable Code:

```
className = " " + selector + " ";
```

Copy/Paste Fix:

```
const statement = db.prepare('SELECT column FROM table WHERE id = ?'); statement.get(userValue);
```

Remediation Strategy: Use parameterized prepared statements instead of dynamic SQL string concatenation.

References:

<https://cwe.mitre.org/data/definitions/89.html>

https://cheatsheetseries.owasp.org/cheatsheets/Query_Parameterization_Cheat_Sheet.html